

PART 5 OF 5. TEST THE CONTROLS AND DECIDE WHO HANDLES FAILURES.

Know when to stop the automation

Before you schedule the pilot, test what happens when the data is incomplete, a reply can't be matched, or a message might have been sent twice. Use fictional records and test accounts for those checks.

This part gives you a failure checklist and a short operating record. Keep them with the script so the person maintaining it knows what can run, what must stop, and who needs to hear about a problem.

1 Keep a reply separate from the state of the work

A reply tells you what the sender wrote. "Haven't started," "this isn't mine," and "waiting for access" all deserve attention, but none means the item is now in progress.

For this pilot, let the automation propose a comment containing the owner's exact words. Leave state and due-date changes to the normal tracker workflow. If you record that a matched reply arrived, keep that as reminder history, separate from the work item's status.

EXAMPLE

An owner replies "Done with my part." Keep that sentence intact. Before anyone closes the work item, confirm whether the item includes other people's work or a remaining review.

If you later automate a state change, define the precise action that authorizes it, who can take that action, and which states it can change. Test the transition separately from copying a comment.

WHY THIS MATTERS: a wrong state in the tracker is survivable while the person's own words sit next to it. That state gets copied into a report later, and the words don't travel with it.

2 Refuse ambiguous replies and duplicate writes

Use the reply matching from Part 4: source message reference, item ID, and authorized sender. If any of those checks fails, record the reason and leave the work item unchanged.

Don't add an override that makes a failed match look successful. A person can resolve the question outside the automation, open the right item, and record the update through the normal workflow.

EXAMPLE

Someone sends "What is this?" as a new chat message. With no matching source reference, the automation puts it in the review list and makes no tracker comment. The pilot owner answers the question manually.

Use stored reply event IDs to recognize repeats. Prevent two overlapping runs from processing the same event at once. Have the implementation reviewer check how that protection behaves after a restart.

A request can time out after the tracker has accepted it. If the API doesn't provide a reliable way to retry without creating another comment, hold that write and reconcile it with the tracker. Don't count an uncertain result as either confirmed success or confirmed failure.

WHY THIS MATTERS: an override that lets an unmatched reply through gets used on a Friday afternoon when somebody wants the queue cleared. A system that refuses and leaves the item alone is easier to trust than one that's usually right.

3 Exercise every control before trusting it

Keep sending and tracker writes disabled by default. Use separate controls for each, and make the running process check them before each external action. A status screen that says "disabled" isn't enough evidence.

Test in an approved test environment. Count the actual messages and tracker changes, not just what the script says it did.

- ☐ Sending off: a qualifying item produces a preview and no message.
- ☐ Writes off: a matched reply produces a proposed comment and no tracker change.
- ☐ Wrong sender or missing reply reference: no write, with a visible reason.
- ☐ Duplicate reply event: at most one comment, including after a restart.
- ☐ Message limit exceeded: the proposed batch is held before the first send.
- ☐ Read fails or returns only part of the scope: the run is incomplete and sends nothing.
- ☐ Stop requested during a run: no new actions begin after the process observes the stop. Record any request already in flight and reconcile its result.

You can't retract a delivered message by stopping the next one. Test how long the stop takes to reach the running process, and tell the pilot owner where to check for actions already completed.

EXAMPLE

Test result to record: "Writes off. Fed one matched reply for DEMO-101. The script logged a proposed comment. Opened DEMO-101 in the tracker and confirmed no comment was added. Passed." Run this again after every change to the reply path.

WHY THIS MATTERS: the real risk is believing a control works and no longer watching the thing it protects. Count the actual messages and tracker changes, because a script that reports "sending disabled" can still have sent them.

4 Make a failed run distinguishable from an empty report

Record the run time, scope, rule version, read result, counts, delivery results, and unresolved actions. A successful run with zero qualifying items should look different from a run that couldn't read the tracker.

Send failures to the pilot owner through an approved alert route. If that route uses the same service as the reminder system, choose another way to notice a missed run, such as a scheduled check of the last-success time. Don't put credentials or unnecessary message content in logs.

EXAMPLE

"Run at 9:00 AM failed because tracker access expired. No messages sent. Last successful read: yesterday at 9:00 AM. Action: restore approved access and rerun the preview before enabling delivery."

If delivery fails halfway through, show which messages are confirmed sent, confirmed unsent, or uncertain. Don't restart the whole batch and hope duplicate protection covers it. Reconcile uncertain sends first.

Count only confirmed delivered reminders when tracking unanswered requests, and describe the result as "no matched reply recorded." That doesn't prove the person ignored the request. Agree on reminder intervals and when a person should follow up manually. Don't automatically notify a manager because a counter reached an arbitrary number.

WHY THIS MATTERS: a run that sends nothing because the work is on track and a run that sends nothing because access expired look identical from outside. Without that distinction you'll read the quiet week as good news.

5**Name the owner before adding a schedule or dashboard**

The pilot needs someone who can pause it, investigate failures, and explain what it changed. Write these decisions down before leaving it unattended:

- ☐ Pilot owner and backup contact:
- ☐ Approved project scope and participants:
- ☐ Schedule and time zone:
- ☐ Sending and write controls, including how to stop a running job:
- ☐ Message limits and minimum reminder interval:

- ☐ Failure alert route and missed-run check:
- ☐ Where pending replies and uncertain actions are reviewed:
- ☐ Approved storage location and retention period:
- ☐ Date to review whether the pilot is useful:

At the review, compare the time spent running and correcting the tool with the manual task you measured in Part 1. Ask participants whether the reminders help, whether the timing works, and which messages they didn't need.

If you add a dashboard, show the last successful read, the rules behind each count, and the number of items held for review. Keep incomplete or old data labeled. Don't infer progress from reminder activity or label a state as blocked unless the source supports it.

EXAMPLE

Fictional operating record: "Pilot owner Alex Example, backup Sam Example. Scope: the Setup project, four participants. Runs 9:00 AM weekdays, America/Los_Angeles. Stop: set both controls off and cancel the scheduled job. Review date: November 3."

WHY THIS MATTERS: an unattended tool with no named owner keeps running after the person who built it moves teams. Write down who can stop it while you still remember how.

Keep the test results and operating record with the script. Rerun the relevant tests whenever you change a rule, permission, field mapping, or message path.

Expand beyond the pilot only after you've resolved incorrect recipients, ambiguous writes, and missed failures, and the next participants have agreed to the same behavior.